LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Shatrya Christiano Chandra 0706012410064

Bagianmu: Representasi papan, logika aturan, DFS (flood fill), penyatuan evaluasi, playtest

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027
Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 29 Oktober 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Representasi Papan dan Setup Proyek
7. Logika Aturan (rules.js)
8. DFS: Depth-First Search dan Flood Fill
9. Menyatukan BFS dan DFS di evaluate()
10. Pengujian Aturan
11. Panduan Playtest (Tabel 2.2.3, baris 4)
12. Hasil Pengujian (Proposal Bab 2.2)
13. Fitur Rally: Poin, Leaderboard, Admin
14. Latihan
15. Pertanyaan Presentasi yang Mungkin Muncul
16. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan representasi papan (array 49 kotak) dan cara mengubah koordinat a1–g7 menjadi indeks.
- Menjelaskan setiap fungsi aturan di `rules.js` dan bagaimana game memastikan langkah sah.
- Menjelaskan DFS/flood fill dan cara mendeteksi papan yang sudah terbelah.
- Menjelaskan bagaimana BFS dan DFS disatukan di `evaluate()`.
- Menjalankan playtest dan mencatat hasilnya dengan benar.

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
1	Setup proyek HTML/CSS/JS, representasi papan	`index.html`, `config.js`, `createState()`
2	Logika aturan: lompatan legal, bakar, jebakan, cek kalah	`rules.js`; `game.js`: `handleCell()`
2–3	DFS (flood fill): cek papan terbelah, luas area	`ai.js`: `dfs()`
3	Penyatuan BFS + DFS ke evaluasi (bersama Clarrence)	`ai.js`: `evaluate()`
3	Playtest ±10 orang, kuesioner, perbaikan bug	Lampiran di guide ini

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak terbuka.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, batas AI, poin	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, animasi, game over	Shatrya & Louis
`js/board.js`, `js/fx.js`, `css/style.css`, `index.html`	Tampilan papan, knight, efek api/jebakan, layout	Clarrence
`js/main.js`, `js/i18n.js`, `js/audio.js`	Layar menu, pengaturan, panel statistik, teks dua bahasa, suara	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`js/scoring.js`, `api/*`, `admin.html`	Poin, leaderboard, halaman admin (fitur rally)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, mode online, dan server leaderboard semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar, jebakan, atau kotak lawan	`isOpen()` dipakai oleh `legalJumps()`
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 53–55

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 57–61

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (isOpen(s, n)) out.push(n);
  return out;
}
```

js/rules.js : baris 74–80

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 83–85

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

js/rules.js : baris 111–113

```
111  export function winner(s) {
112    return canMove(s, s.turn) ? null : s.turn ^ 1;
113  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 119–136

```
119   nextTurn() {
120     if (this.disposed) return;
121     const w = winner(this.state);
122     if (w !== null) return this.finish(w);
123     const p = this.current;
124     if (p.kind === 'human') {
125       this.phase = 'jump';
126       if (this.mode === 'local' && this.history.length) sfx.turn();
127       this.refresh();
128     } else if (p.kind === 'ai') {
129       this.phase = 'ai';
130       this.refresh();
131       this.runAi();
132     } else {
133       this.phase = 'remote';
134       this.refresh();
135     }
136   }
```

js/game.js : baris 138–162

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, animasi dan tombol akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 78–108

```
78  export function evaluate(s, me) {
79    const { cells, pos, turn } = s;
80    const areaA = dfs(cells, pos, 0, markA);
81    const areaB = dfs(cells, pos, 1, markB);
82    let split = true;
83    for (let i = 0; i < SIZE; i++) {
84      if (markA[i] === 1 && markB[i] === 1) {
85        split = false;
86        break;
87      }
88    }
89    let score;
90    if (split) {
91      score = (areaA - areaB) * SPLIT_WEIGHT;
92    } else {
93      bfs(cells, pos, 0, distA);
94      bfs(cells, pos, 1, distB);
95      let terr = 0;
96      for (let i = 0; i < SIZE; i++) {
97        if (!open(cells, pos, i)) continue;
98        const a = distA[i];
99        const b = distB[i];
100       if (a < 0 && b < 0) continue;
101       if (b < 0 || (a >= 0 && a < b)) terr++;
102       else if (a < 0 || b < a) terr--;
103       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
104     }
105     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
106   }
107   return me === 0 ? score : -score;
108 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100`.
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas`. Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Representasi Papan dan Setup Proyek

Struktur proyek tanpa framework

Game ini HTML, CSS, dan JavaScript murni (ES modules), tanpa framework dan tanpa build step. `index.html` memuat satu script: `<script type="module" src="js/main.js">`; sisanya dimuat lewat `import`. Karena AI berjalan di *module Web Worker*, halaman harus dibuka lewat server (misalnya `npx serve .`), tidak bisa dengan klik ganda file.

Dari koordinat ke indeks

Kotak	Baris (r)	Kolom (c)	Indeks = r × 7 + c
g7 (kanan atas)	0	6	6
d4 (tengah)	3	3	24
a1 (kiri bawah)	6	0	42

Kebalikannya: `r = Math.floor(i / 7)`, `c = i % 7`. Nama kotak dibuat oleh `squareName()`:

js/rules.js : baris 115–117

```
115  export function squareName(sq) {
116    return 'abcdefg'[sq % N] + (N - Math.floor(sq / N));
117  }
```

js/rules.js : baris 42–44

```
42  export function serialize(s) {
43    return { cells: Array.from(s.cells), pos: [s.pos[0], s.pos[1]], turn: s.turn, moves: s.moves };
44  }
```

js/rules.js : baris 46–51

```
46  export function deserialize(o) {
47    if (!o || !Array.isArray(o.cells) || o.cells.length !== SIZE || !Array.isArray(o.pos)) {
48      throw new Error('bad state');
49    }
50    return { cells: Int8Array.from(o.cells), pos: [o.pos[0] | 0, o.pos[1] | 0], turn: o.turn & 1, moves:
o.moves | 0 };
51  }
```

`Int8Array` tidak bisa dikirim langsung lewat jaringan atau ke worker sebagai JSON biasa, jadi `serialize` mengubahnya ke array biasa. `deserialize` juga **memeriksa** data (panjang harus 49, `pos` harus array) dan melempar error kalau rusak. Ini penting untuk mode online, karena data dari pemain lain tidak boleh dipercaya begitu saja. `| 0` dan `& 1` memaksa nilai menjadi bilangan bulat dan 0/1.

7 Logika Aturan (rules.js)

js/rules.js : baris 63–66

```
63  export function canMove(s, side = s.turn) {
64    for (const n of NEIGHBORS[s.pos[side]]) if (isOpen(s, n)) return true;
65    return false;
66  }
```

js/rules.js : baris 68–71

```
68  export function hasOpenSquare(s) {
69    for (let i = 0; i < SIZE; i++) if (isOpen(s, i)) return true;
70    return false;
71  }
```

js/rules.js : baris 87–90

```
87  export function endTurn(s) {
88    s.turn ^= 1;
89    s.moves++;
90  }
```

`canMove` sama seperti `legalJumps(...).length > 0`, tetapi berhenti di lompatan legal pertama, jadi lebih cepat. `hasOpenSquare` dipakai untuk kasus langka: setelah lompat, tidak ada kotak terbuka yang bisa dijebak. Saat itu jebakan dilewati dan dicatat sebagai `trap = -1`.

Validasi langkah lengkap

js/rules.js : baris 93–101

```
93  export function validateMove(s, move) {
94    if (!move || !Number.isInteger(move.jump) || !Number.isInteger(move.trap)) return false;
95    if (winner(s) !== null) return false;
96    if (!legalJumps(s).includes(move.jump)) return false;
97    const t = cloneState(s);
98    applyJump(t, move.jump);
99    if (!hasOpenSquare(t)) return move.trap === -1;
100   return move.trap >= 0 && move.trap < SIZE && isOpen(t, move.trap);
101 }
```

1. `jump` dan `trap` harus bilangan bulat (menolak data aneh dari jaringan).
2. Game belum selesai.
3. Lompatan termasuk `legalJumps`.
4. Lompatan diterapkan ke **salinan** (`cloneState`), supaya state asli tidak berubah.
5. Kalau tidak ada kotak terbuka, jebakan harus −1. Kalau ada, jebakan harus kotak terbuka (bukan kotak knight mana pun, bukan kotak yang baru terbakar).

Fungsi ini dipakai di tiga tempat: host online memeriksa langkah tamu, game memeriksa langkah AI sebelum dimainkan, dan server leaderboard memutar ulang seluruh game.

js/rules.js : baris 103–108

```
export function applyMove(s, move) {
  const from = applyJump(s, move.jump);
  applyTrap(s, move.trap);
  endTurn(s);
  return from;
}
```

Input pemain: dua klik per giliran

js/game.js : baris 191–230

```
  async handleCell(sq) {
    if (this.disposed || this.current?.kind !== 'human') return;
    const s = this.state;
    const side = s.turn;
    if (this.phase === 'jump') {
      if (!legalJumps(s).includes(sq)) {
        if (sq !== s.pos[side]) sfx.invalid();
        return;
      }
      this.phase = 'animating';
      const from = applyJump(s, sq);
      this.pendingFrom = from;
      this.pendingJump = sq;
      this.lastLand = sq;
      this.lastTrap = null;
      sfx.jump();
      const anim = this.board.animateJump(side, from, sq);
      this.refresh();
      this.board.burn(from);
      sfx.burn();
      await anim;
      if (this.disposed) return;
      if (!hasOpenSquare(s)) return this.completeHumanMove(-1);
      this.phase = 'trap';
      this.refresh();
    } else if (this.phase === 'trap') {
      if (!isOpen(s, sq)) {
        sfx.invalid();
        return;
      }
      this.phase = 'animating';
      applyTrap(s, sq);
      this.lastTrap = sq;
      this.refresh();
      sfx.trap();
      await this.board.trap(sq);
      if (this.disposed) return;
      this.completeHumanMove(sq);
    }
  }
```

Fase `jump`: klik di kotak yang bukan lompatan legal memutar suara "invalid". Kalau legal, `applyJump` dijalankan, kotak asal disimpan di `pendingFrom` (untuk tombol batal), dan animasi diputar. Selama animasi, fase menjadi `animating` sehingga klik diabaikan. **Fase `trap`:** klik harus di kotak terbuka; lalu `applyTrap` dan `completeHumanMove` mengakhiri giliran.

js/game.js : baris 232–250

```
  async undoJump() {
    if (this.phase !== 'trap' || this.pendingFrom == null) return;
    const s = this.state;
    const side = s.turn;
    const at = s.pos[side];
    this.phase = 'animating';
    s.pos[side] = this.pendingFrom;
    s.cells[this.pendingFrom] = EMPTY;
    this.lastLand = null;
    const anim = this.board.animateJump(side, at, this.pendingFrom);
    this.refresh();
    sfx.select();
    await anim;
    if (this.disposed) return;
    this.pendingFrom = null;
    this.pendingJump = null;
    this.phase = 'jump';
    this.refresh();
  }
```

Batal lompat aman karena giliran baru dihitung setelah jebakan dipasang. Knight dikembalikan, dan kotak asal dibuat `EMPTY` lagi (tidak jadi terbakar).

8 DFS: Depth-First Search dan Flood Fill

Ide

DFS menelusuri satu jalur **sedalam mungkin** sebelum mundur. Struktur datanya **tumpukan** (stack, LIFO): yang terakhir masuk keluar pertama. Di proyek ini DFS dipakai untuk **flood fill**: menandai semua kotak yang bisa dicapai knight lewat rangkaian lompatan, seperti tumpahan cat yang menyebar ke semua kotak yang terhubung.

Untuk flood fill, urutan kunjungan tidak penting (yang penting *semua* kotak terhubung ditemukan), jadi DFS cocok dan sedikit lebih sederhana daripada BFS: tidak perlu menyimpan jarak.

`js/ai.js : baris 57–75`

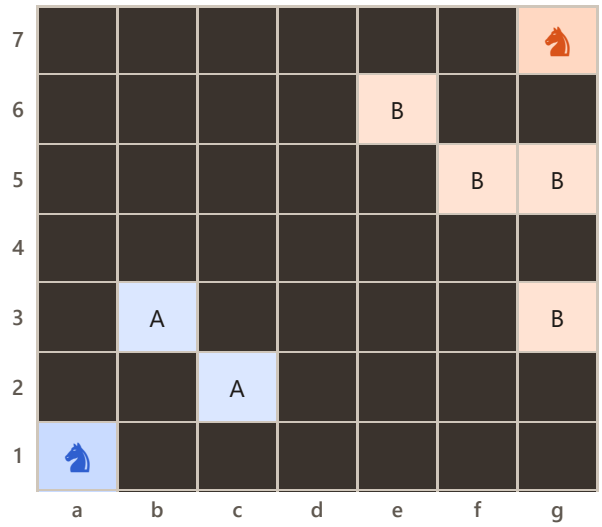
```
57  export function dfs(cells, pos, side, mark) {
58    mark.fill(0);
59    const start = pos[side];
60    mark[start] = 2;
61    let top = 0;
62    let size = 0;
63    stack[top++] = start;
64    while (top > 0) {
65      const cur = stack[--top];
66      for (const n of NEIGHBORS[cur]) {
67        if (mark[n] === 0 && open(cells, pos, n)) {
68          mark[n] = 1;
69          size++;
70          stack[top++] = n;
71        }
72      }
73    }
74    return size;
75  }
```

Baris	Arti
`mark.fill(0)`	Semua kotak belum ditandai.
`mark[start] = 2`	Kotak awal diberi tanda khusus 2 supaya tidak ikut dihitung sebagai area dan tidak dikunjungi lagi.
`stack[top++] = start`	Push kotak awal ke tumpukan.
`stack[--top]`	Pop: ambil kotak paling atas.
`mark[n] === 0 && open(...)`	Tetangga belum ditandai dan masih terbuka → tandai 1, tambah luas, push.
`return size`	Luas area = jumlah kotak yang bisa dicapai (kotak awal tidak dihitung).

> **Kenapa stack berukuran `SIZE * 8`?** Ukuran aman: setiap kotak hanya di-push sekali (karena langsung ditandai), jadi 49 sebenarnya cukup, tetapi buffer lebih besar tidak berbahaya dan dibuat sekali saja.

Mendeteksi papan terbelah

DFS dijalankan dari kedua knight, dengan tanda di `markA` dan `markB`. Kalau tidak ada satu pun kotak yang ditandai 1 di keduanya, kedua knight hidup di area terpisah dan tidak bisa saling mengganggu lagi. Pemenangnya ditentukan oleh siapa yang areanya lebih luas (dia bisa melompat lebih lama).



Contoh papan terbelah (dipakai di tests/run.mjs). Area P1 = 2 kotak, area P2 = 4 kotak, tidak ada kotak yang sama → split. Skor P2 = (4 − 2) × 100 = 200.

js/ai.js : baris 78–91

```
78  export function evaluate(s, me) {
79    const { cells, pos, turn } = s;
80    const areaA = dfs(cells, pos, 0, markA);
81    const areaB = dfs(cells, pos, 1, markB);
82    let split = true;
83    for (let i = 0; i < SIZE; i++) {
84      if (markA[i] === 1 && markB[i] === 1) {
85        split = false;
86        break;
87      }
88    }
89    let score;
90    if (split) {
91      score = (areaA - areaB) * SPLIT_WEIGHT;
```

9 Menyatukan BFS dan DFS di evaluate()

js/ai.js : baris 78–108

```
78  export function evaluate(s, me) {
79    const { cells, pos, turn } = s;
80    const areaA = dfs(cells, pos, 0, markA);
81    const areaB = dfs(cells, pos, 1, markB);
82    let split = true;
83    for (let i = 0; i < SIZE; i++) {
84      if (markA[i] === 1 && markB[i] === 1) {
85        split = false;
86        break;
87      }
88    }
89    let score;
90    if (split) {
91      score = (areaA - areaB) * SPLIT_WEIGHT;
92    } else {
93      bfs(cells, pos, 0, distA);
94      bfs(cells, pos, 1, distB);
95      let terr = 0;
96      for (let i = 0; i < SIZE; i++) {
97        if (!open(cells, pos, i)) continue;
98        const a = distA[i];
99        const b = distB[i];
100       if (a < 0 && b < 0) continue;
101       if (b < 0 || (a >= 0 && a < b)) terr++;
102       else if (a < 0 || b < a) terr--;
103       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
104     }
105     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
106   }
107   return me === 0 ? score : -score;
108 }
```

1. **DFS dulu** untuk kedua knight, hitung luas area.
2. **Cek split:** apakah ada kotak yang dicapai keduanya?
3. **Kalau terbelah:** skor = (areaA − areaB) × `SPLIT_WEIGHT` (100).
4. **Kalau belum:** BFS dari kedua knight, hitung wilayah (bagian Clarrence), tambah 0,5 × selisih mobilitas.
5. **Sudut pandang:** skor dihitung untuk P1, lalu tandanya dibalik kalau AI adalah P2.

> **Kenapa bobot 100?** Skor wilayah paling besar sekitar 49 + 4 (mobilitas) ≈ 53. Dengan × 100, selisih area 1 kotak pada papan terbelah (100) selalu lebih besar daripada keunggulan wilayah apa pun. Artinya AI lebih suka posisi yang *sudah pasti* menang daripada posisi terbuka yang hanya kelihatan bagus. Tes memeriksa ini: `SPLIT_WEIGHT > 49 + 4`.

Urutan DFS → BFS juga menghemat waktu: kalau papan sudah terbelah, dua BFS tidak perlu dijalankan.

10 Pengujian Aturan

tests/run.mjs : baris 38–108

```
38  // ---------------------------------------------------------------------------
39  console.log('\n1. Rules');
40  {
41    const s = createState();
42    check('start: P1 on a1', s.pos[0] === sq('a1'));
43    check('start: P2 on g7', s.pos[1] === sq('g7'));
44    check('start: P1 moves first', s.turn === 0);
45    check('a1 legal jumps = b3, c2', JSON.stringify(legalJumps(s).sort((a, b) => a - b)) ===
JSON.stringify([sq('b3'), sq('c2')].sort((a, b) => a - b)));
46
47    const t = cloneState(s);
48    const from = applyJump(t, sq('b3'));
49    check('jump moves knight', t.pos[0] === sq('b3'));
50    check('origin square burns', from === sq('a1') && t.cells[sq('a1')] === BURNED);
51    applyTrap(t, sq('d4'));
52    check('trap marks square', t.cells[sq('d4')] === TRAP);
53    endTurn(t);
54    check('turn passes', t.turn === 1 && t.moves === 1);
55
56    // Cannot land on burned, trap or occupied squares.
57    const u = createState({ p1: sq('d4'), p2: sq('e6'), first: 0 });
58    u.cells[sq('b3')] = BURNED;
59    u.cells[sq('c6')] = TRAP;
60    const j = legalJumps(u);
61    check('blocked: burned square', !j.includes(sq('b3')));
62    check('blocked: trap square', !j.includes(sq('c6')));
63    check('blocked: opponent square', !j.includes(sq('e6')));
64    check('open squares still legal', j.includes(sq('f5')) && j.includes(sq('b5')));
65
66    // Trap validation.
67    check('valid move accepted', validateMove(u, { jump: sq('f5'), trap: sq('a1') }));
68    check('trap on burned rejected', !validateMove(u, { jump: sq('f5'), trap: sq('b3') }));
69    check('trap on opponent rejected', !validateMove(u, { jump: sq('f5'), trap: sq('e6') }));
70    check('trap on own new square rejected', !validateMove(u, { jump: sq('f5'), trap: sq('f5') }));
71    check('trap on burned origin rejected', !validateMove(u, { jump: sq('f5'), trap: sq('d4') }));
72    check('illegal jump rejected', !validateMove(u, { jump: sq('d5'), trap: sq('a1') }));
73    check('missing trap rejected while squares are open', !validateMove(u, { jump: sq('f5'), trap: -1
}));
74    check('garbage rejected', !validateMove(u, { jump: 'x', trap: 3 }) && !validateMove(u, null));
75
76    // Loss detection: P1 on a1 with b3 and c2 gone loses on its turn.
77    const v = createState();
78    v.cells[sq('b3')] = TRAP;
79    v.cells[sq('c2')] = BURNED;
80    check('no legal jump -> side to move loses', winner(v) === 1 && !canMove(v));
81    v.turn = 1;
82    check('other side still alive', winner(v) === null);
83
84    // Trap skipped only when nothing is open.
85    const w = createState({ p1: sq('a1'), p2: sq('g7'), first: 0 });
86    for (let i = 0; i < 49; i++) if (i !== sq('a1') && i !== sq('g7') && i !== sq('b3')) w.cells[i] =
BURNED;
87    check('no open square after jump -> trap -1 valid', validateMove(w, { jump: sq('b3'), trap: -1 }));
88    check('no open square after jump -> real trap invalid', !validateMove(w, { jump: sq('b3'), trap:
sq('c5') }));
89
90    const r = deserialize(JSON.parse(JSON.stringify(serialize(u))));
```

```
  check('serialize round trip', r.pos[0] === u.pos[0] && r.cells[sq('c6')] === TRAP && r.turn === u.turn);
  let threw = false;
  try { deserialize({ cells: [1, 2], pos: [0, 1] }); } catch { threw = true; }
  check('deserialize rejects bad payload', threw);

  // Evaluation sanity.
  const e = createState();
  check('start position is symmetric (P1 gets tie squares)', evaluate(e, 0) === -evaluate(e, 1));
  // Split board: P1 can only reach b3, c2; P2 can reach f5, e6, g3, g5.
  const split = createState({ p1: sq('a1'), p2: sq('g7'), first: 0 });
  const keep = new Set(['a1', 'g7', 'b3', 'c2', 'f5', 'e6', 'g3', 'g5'].map(sq));
  for (let i = 0; i < 49; i++) if (!keep.has(i)) split.cells[i] = BURNED;
  check('DFS split: area difference x weight', evaluate(split, 1) === 2 * SPLIT_WEIGHT, String(evaluate(split, 1)));
  check('split weight outranks territory', SPLIT_WEIGHT > 49 + 4);
  // BFS territory: P1 to move on an open board gets the tie squares.
  const terr = createState({ p1: sq('d4'), p2: sq('g7'), first: 0 });
  check('BFS territory favors the central knight', evaluate(terr, 0) > 0);
}
```

Bagian 1 dari tes memeriksa setiap aturan: posisi awal, lompatan legal dari a1 (hanya b3 dan c2), kotak asal terbakar, jebakan tercatat, giliran berpindah, tidak bisa mendarat di kotak terbakar/jebakan/lawan, validasi jebakan, deteksi kalah, kasus tidak ada kotak tersisa, serialisasi, dan contoh papan terbelah untuk DFS.

11 Panduan Playtest (Tabel 2.2.3, baris 4)

Target proposal: sekitar 10 siswa SMA atau teman, kuesioner skala 1–5, **rata-rata kejelasan aturan minimal 4** dan tingkat tantangan dinilai sesuai. Tes otomatis tidak bisa menggantikan ini, dan hasilnya hanya boleh dilaporkan kalau benar-benar dilakukan.

Langkah

1. Jelaskan aturan maksimal 1 menit (pakai layar "Cara Bermain"). Catat waktunya.
2. Peserta main 1 game di Easy, lalu 1 game di level pilihannya.
3. Jangan membantu saat bermain; catat di mana peserta bingung.
4. Isi kuesioner, lalu tanyakan satu hal yang paling membingungkan.

Kuesioner (skala 1 = sangat tidak setuju, 5 = sangat setuju)

No	Pernyataan	1	2	3	4	5
1	Aturan game mudah dipahami.					
2	Saya tahu kotak mana yang boleh saya lompati.					
3	Saya mengerti kenapa saya menang atau kalah.					
4	Tingkat kesulitan yang saya pilih terasa menantang tapi adil.					
5	AI terasa "berpikir", bukan bergerak acak.					
6	Saya ingin main lagi.					

Catatan bug

No	Apa yang terjadi	Langkah untuk mengulang	Perangkat / browser	Status
1				
2				
3				

Laporkan: jumlah peserta, rata-rata tiap pernyataan, level yang dipilih, berapa yang menang, dan bug yang ditemukan beserta perbaikannya.

12 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **927 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	872	872	0,0%	0	10	6
2	30/30	30/30	24.540	7.050	71,3%	781	91	26
3	30/30	30/30	739.226	37.689	94,9%	2.242	2.063	191

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 20 posisi yang diperiksa Minimax biasa, ±10× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	100%	57%
Medium	2	100%	74%
Hard	3	100%	93%
Expert	4	100%	86%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (semua level 100%). Melawan greedy, win rate naik dari Easy ke Hard, tetapi **Expert (86%) sedikit di bawah Hard (93%)**. Jadi target "win rate naik seiring level" hanya tercapai sebagian. Jawaban jujur kalau ditanya:

- Dengan 100 game, selisih 7 poin persen masih dalam rentang acak (pemain greedy memilih acak di antara langkah yang sama bagusnya).
- Fungsi evaluasi BFS/DFS adalah perkiraan. Melihat lebih dalam dengan evaluasi yang tidak sempurna tidak selalu lebih baik (*horizon effect*).
- Pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus di kedalaman lebih besar.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	931	0,1 ms	0,3 ms	0,4 ms
Medium	934	1,5 ms	4,4 ms	12,1 ms
Hard	914	18,5 ms	41,5 ms	82,7 ms
Expert	915	101,1 ms	316,3 ms	626,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Otak AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

13 Fitur Rally: Poin, Leaderboard, Admin

Fitur ini **tidak ada di proposal**; ditambahkan supaya game bisa dipakai di rally NPLC. Bukan bagian dari empat algoritma AI, tetapi mungkin ditanya saat demo.

Level	Pengali	Per langkah	Bonus menang
Easy	×1	+10	+200
Medium	×1,5	+15	+300
Hard	×2	+20	+400
Expert	×2,5	+25	+500

Hanya mode Single Player (lawan AI) yang memberi poin. Contoh: menang di Hard dengan 12 langkah = 12 × 20 + 400 = **640 poin**.

js/scoring.js : baris 64–96

```
64  export function scoreGame(moves, { mode = 'ai', level, humanSide, rules = SCORING }) {
65    const mult = rules.multiplier?.[level] ?? 0;
66    const out = {
67      valid: true, badMove: -1, finished: false, won: false, mult, steps: [],
68      humanMoves: 0, movePoints: 0, winPoints: 0, total: 0,
69    };
70    if (!pointsEnabled(mode) || !Object.hasOwn(DIFFICULTY, level) || !Array.isArray(moves)) return out;
71
72    const per = levelValues(rules, level);
73    const s = createState();
74    for (let i = 0; i < moves.length; i++) {
75      const m = moves[i];
76      if (!validateMove(s, m)) {
77        out.valid = false;
78        out.badMove = i;
79        return out;
80      }
81      const side = s.turn;
82      applyJump(s, m.jump);
83      applyTrap(s, m.trap);
84      if (side === humanSide) out.humanMoves++;
85      out.steps.push(side === humanSide ? per.move : 0);
86      endTurn(s);
87    }
88
89    const w = winner(s);
90    out.finished = w !== null;
91    out.won = w === humanSide;
92    out.movePoints = out.humanMoves * per.move;
93    if (out.won) out.winPoints = per.win;
94    out.total = out.movePoints + out.winPoints;
95    return out;
96  }
```

- **Anti curang:** browser hanya mengirim daftar langkah. Server (`api/score.js`) memutar ulang semua langkah dengan `rules.js` yang sama, menolak langkah ilegal atau game yang belum selesai, lalu menghitung poin sendiri.

- **Leaderboard:** total poin per tim dari semua game, urut total → jumlah menang → yang lebih dulu mengirim. Disimpan di Upstash Redis lewat Vercel.
- **Admin:** `/admin.html` dengan kunci `ADMIN_KEY` bisa mengubah poin per langkah, bonus menang, dan pengali tiap level.

14 Latihan

Latihan 1: Indeks

Berapa indeks e2 dan f5? Kotak apa yang berindeks 17?

Jawaban: e2: r = 7 − 2 = 5, c = 4 → 39. f5: r = 2, c = 5 → 19. Indeks 17: r = 2, c = 3 → d5.

Latihan 2: Validasi

Knight P1 di d4, P2 di e6, kotak b3 terbakar. Apakah langkah `{ jump: f5, trap: d4 }` sah?

Jawaban: Tidak. Lompatan ke f5 legal, tetapi setelah lompat, d4 terbakar, jadi tidak bisa dijebak. Tes "trap on burned origin rejected" memeriksa kasus ini.

Latihan 3: Flood fill

Pada contoh papan terbelah, area P1 = {b3, c2} dan P2 = {e6, f5, g3, g5}. Kenapa P1 tidak bisa mencapai f5 walaupun f5 terbuka?

Jawaban: Semua jalur dari b3/c2 ke f5 melewati kotak terbakar. DFS hanya menyebar ke tetangga yang terbuka, jadi f5 tidak pernah dicapai dari a1.

15 Pertanyaan Presentasi yang Mungkin Muncul

T: Kenapa papan disimpan sebagai array satu dimensi, bukan 7 × 7?

J: Lebih cepat dan sederhana: satu angka per kotak, mudah disalin (`Int8Array`), dan tetangga knight sudah dihitung sebagai daftar indeks.

T: Kenapa DFS untuk flood fill, bukan BFS?

J: Untuk flood fill hanya perlu tahu kotak mana yang terhubung, bukan jaraknya. Keduanya benar; DFS dengan stack sedikit lebih sederhana. Proposal memakai keduanya untuk tujuan berbeda: BFS untuk jarak/wilayah, DFS untuk area/split.

T: Bagaimana game mencegah langkah curang?

J: Semua langkah melewati `legalJumps` / `isOpen`; langkah dari AI, pemain online, dan server diperiksa dengan `validateMove`. Aturan hanya ditulis sekali di `rules.js`.

T: Apa arti "papan terbelah" dan kenapa skornya × 100?

J: Kedua knight tidak lagi bisa mencapai kotak yang sama, jadi hasil game praktis ditentukan oleh luas area. × 100 memastikan situasi ini selalu dinilai lebih penting daripada skor wilayah (maks. ±53).

T: Apa yang terjadi kalau tidak ada kotak untuk dijebak?

J: Jebakan dilewati dan dicatat sebagai −1. `validateMove` hanya menerima −1 dalam kasus ini.

T: Apa hasil playtest kalian?

J: Laporkan angka nyata dari kuesioner. Kalau playtest belum selesai, katakan itu dengan jujur dan jelaskan rencananya.

16 Checklist Sebelum Presentasi

- ☐ Bisa mengubah koordinat ke indeks dan sebaliknya tanpa kalkulator.
- ☐ Bisa menunjuk fungsi untuk setiap aturan (a)–(e) di `rules.js`.
- ☐ Bisa menjelaskan flood fill dengan contoh papan terbelah.
- ☐ Bisa menjelaskan urutan DFS → BFS di `evaluate()` dan alasan bobot 100.
- ☐ Data playtest sudah direkap (jumlah peserta, rata-rata, bug).
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.